LokRozgar_AI Backend API Documentation (v1)

Backend API Reference (v1)

Complete REST endpoint reference for the LokRozgar AI backend service, including access control and request semantics.

PROJECT	LokRozgar AI
DOCUMENT	API Documentation
API VERSION	v1
BASE URL	`http://localhost:8000/api/v1`
TECHNICAL SCOPE	Node.js · Express · MongoDB

This document is generated from the project's Markdown source of truth. Its technical content is reproduced without modification.

Contents

1. Authentication Module (`/auth`) 3

2. User Module (`/users`) 4

3. Worker Profile Module (`/worker-profile`) 5

4. Employer Profile Module (`/employer-profile`) 6

5. Catalog Module (`/catalog`) 7

6. Job Module (`/jobs`) 8

7. Application Module (`/applications`) 9

8. Voice Profile Module (`/voice-profile`) 10

- Quick Flow (single recording → full extraction) 10
- Guided Flow (one question at a time, server-side draft) 10
- Shared Commit Path (both flows) 10

9. Worker Skills Module (`/worker-skills`) 11

10. Resume Module (`/resumes`) 12

11. Documents Module (`/documents`) 13

12. Saved Jobs Module (`/saved-jobs`) 14

13. Shifts Module (`/shifts`) 15

14. Payments Module (`/payments`) 16

15. Ratings Module (`/ratings`) 17

16. Disputes Module (`/disputes`) 18

17. Chat Module (`/chat`) 19

18. Notifications Module (`/notifications`) 20

19. Admin Module (`/admin`) 21

Appendix A: Employer-Initiated Hiring (formerly `/offers`) 22

Appendix B: Legacy Matching Engine 23

Appendix C: Error Response Format 24

Base URL: `http://localhost:8000/api/v1`

1. Authentication Module (`/auth`)

Endpoint	Method	Access	Description
`/auth/send-otp`	`POST`	Public	Send OTP to phone number
`/auth/verify-otp`	`POST`	Public	Verify phone OTP
`/auth/register`	`POST`	Public	Register new user (worker or employer)
`/auth/login`	`POST`	Public	Login with phone & password
`/auth/logout`	`POST`	Authenticated	Logout user session
`/auth/refresh-token`	`POST`	Public	Refresh JWT access token
`/auth/change-password`	`POST`	Authenticated	Change current user password
`/auth/reset-password`	`POST`	Public	Reset password with OTP verification
`/auth/sessions`	`GET`	Authenticated	List active user sessions
`/auth/sessions/:sessionId/revoke`	`POST`	Authenticated	Revoke specific session
`/auth/add-role`	`POST`	Authenticated	Add additional role (worker/employer)

2. User Module (`/users`)

Endpoint	Method	Access	Description
`/users/me`	`GET`	Authenticated	Fetch current user & profile details
`/users/language`	`PATCH`	Authenticated	Update preferred language ('mr', 'hi', 'en')
`/users/account`	`DELETE`	Authenticated	Delete user account & associated profile data
`/users/device-token`	`POST`	Authenticated	Register FCM device token for push notifications

3. Worker Profile Module (`/worker-profile`)

Endpoint	Method	Access	Description
`/worker-profile/me`	`GET`	Worker/Admin	Fetch worker profile
`/worker-profile`	`POST`	Worker/Admin	Create or update worker profile details
`/worker-profile/availability`	`PATCH`	Worker/Admin	Toggle availability ('available', 'busy', 'unavailable')
`/worker-profile/location`	`PATCH`	Worker/Admin	Update worker location [longitude, latitude]
`/worker-profile/radius`	`PATCH`	Worker/Admin	Update preferred work radius in km
`/worker-profile/categories`	`PATCH`	Worker/Admin	Update preferred job category IDs
`/worker-profile/photo`	`POST`	Worker/Admin	Upload profile photo to Cloudinary

4. Employer Profile Module (`/employer-profile`)

Endpoint	Method	Access	Description
`/employer-profile/me`	`GET`	Employer/Admin	Fetch employer profile
`/employer-profile`	`POST`	Employer/Admin	Create or update employer profile details
`/employer-profile/photo`	`POST`	Employer/Admin	Upload business photo to Cloudinary
`/employer-profile/business`	`PATCH`	Employer/Admin	Update business details

5. Catalog Module (`/catalog`)

Endpoint	Method	Access	Description
`/catalog/categories`	`GET`	Public	List categories with language resolution (`?lang=mr/hi/en`)
`/catalog/skills`	`GET`	Public	List skills with language resolution
`/catalog/categories`	`POST`	Admin	Create job category
`/catalog/categories/:categoryId`	`PATCH`	Admin	Update job category
`/catalog/categories/:categoryId`	`DELETE`	Admin	Deactivate job category
`/catalog/skills`	`POST`	Admin	Create skill
`/catalog/skills/:skillId`	`PATCH`	Admin	Update skill
`/catalog/skills/:skillId`	`DELETE`	Admin	Deactivate skill

6. Job Module (`/jobs`)

Endpoint	Method	Access	Description
`/jobs/search`	`GET`	Public	Search & filter jobs with pagination & language picker
`/jobs/nearby`	`GET`	Public	Fetch nearby jobs using `$geoNear`
`/jobs/recommended`	`GET`	Worker	Fetch AI-recommended jobs for worker (Worker → Jobs)
`/jobs/:jobId/recommended-workers`	`GET`	Employer/ Admin	Fetch AI-recommended workers for a job (Job → Workers)
`/jobs`	`POST`	Employer/ Admin	Create job with auto-translation
`/jobs/employer/mine`	`GET`	Employer/ Admin	List employer's posted jobs
`/jobs/:jobId`	`PATCH`	Employer/ Admin	Update job details
`/jobs/:jobId/cancel`	`POST`	Employer/ Admin	Cancel job
`/jobs/:jobId/close`	`POST`	Employer/ Admin	Mark job completed and closed

`/jobs/recommended` and `/jobs/:jobId/recommended-workers` both call the Python AI service (`../ai-service`) via `services/aiService.client.js` — Node does not compute match scores itself. `/jobs/recommended` response: `[{ job, matchScore, matchFactors }]`, sorted by `matchScore` descending. `/jobs/:jobId/recommended-workers` response: `[{ worker, matchScore, matchFactors }]`, same ordering. `matchFactors.pay` / `.rating` may be `null` (not comparable/not yet rated) rather than `0` — see the AI service's README for why. Both return `503` if the AI service is unreachable, or the AI service's own status code if it rejects the request (e.g. `400` for an invalid `top_k`) — never a fabricated score.

7. Application Module (`/applications`)

Endpoint	Method	Access	Description
`/applications/apply`	`POST`	Worker	Apply for job (scores via the AI service, requires `status: "open"`)
`/applications/:applicationId/withdraw`	`POST`	Worker	Withdraw application
`/applications/mine`	`GET`	Worker	List worker's applications
`/applications/job/:jobId`	`GET`	Employer/ Admin	List applicants for job ranked by score
`/applications/:applicationId/shortlist`	`PATCH`	Employer/ Admin	Shortlist application
`/applications/:applicationId/status`	`PATCH`	Employer/ Admin	Accept/reject application (increments filled count)

`/applications/apply` calls the AI service's `/recommendations/score-pair` via `scoreWorkerJobPair()` and persists both `matchScore` and `matchFactors` on the created `Application` (previously `matchFactors` was computed but not saved - fixed as part of the AI cutover). Only jobs with `status: "open"` accept new applications — a `partially_assigned` job may still appear in `/jobs/recommended` if it needs more workers, but cannot receive a *new* application through this endpoint; this is an intentional difference between recommendation visibility and application eligibility, not an oversight. If the AI service is unreachable, no `Application` is created and the request fails with `503` rather than falling back to a fabricated score.

Legacy matching engine: `services/matching.service.js` (`calculateMatchScore`) is no longer used by any production route. It remains in the repository for reference and is still covered by its own unit test in `tests/api.test.js`, pending an explicit decision to remove it.

8. Voice Profile Module (`/voice-profile`)

All routes require authentication and `worker` or `admin` role.

Quick Flow (single recording → full extraction)

Endpoint	Method	Access	Description
`/voice-profile/extract`	`POST`	Worker/Admin	Upload a single audio recording (multipart `audio` field, ≤15 MB). The backend proxies it to the AI service for STT + LLM extraction. Returns the transcript and all extracted profile fields. No audio is persisted.

Guided Flow (one question at a time, server-side draft)

Endpoint	Method	Access	Description
`/voice-profile/guided/start`	`POST`	Worker/Admin	Start a guided session. Creates a server-side draft keyed by `workerId`. Body: `{ language }`.
`/voice-profile/guided/draft`	`GET`	Worker/Admin	Retrieve the current draft state. Query: `?language=mr`.
`/voice-profile/guided/step`	`POST`	Worker/Admin	Submit a voice recording for one field (multipart `audio`). Transcribes and extracts deterministically for all fields except `skills` (which triggers one LLM call).
`/voice-profile/guided/step`	`PATCH`	Worker/Admin	Set a field by value without audio (tap-only steps: `languages`, `availability`, review-screen edits). Body: `{ field, value }`.
`/voice-profile/guided/draft`	`DELETE`	Worker/Admin	Discard the in-progress draft.

Shared Commit Path (both flows)

Endpoint	Method	Access	Description
`/voice-profile/confirm`	`POST`	Worker/Admin	Commit extracted fields to `WorkerProfile` and upsert `WorkerSkill` documents. Used by both quick and guided flows.

9. Worker Skills Module (`/worker-skills`)

All routes require authentication and `worker` or `admin` role.

Endpoint	Method	Access	Description
`/worker-skills/mine`	`GET`	Worker/ Admin	List current worker's skills with proficiency and verification status
`/worker-skills`	`POST`	Worker/ Admin	Add a skill to worker's profile (by `skillId`)
`/worker-skills/:id`	`PATCH`	Worker/ Admin	Update proficiency level for an existing skill
`/worker-skills/:id`	`DELETE`	Worker/ Admin	Remove a skill from worker's profile
`/worker-skills/:id/verify-request`	`POST`	Worker/ Admin	Request admin verification for a skill
`/worker-skills/:id/evidence`	`POST`	Worker/ Admin	Attach a completed job as evidence for a skill claim

10. Resume Module (`/resumes`)

All routes require authentication and `worker` or `admin` role.

Endpoint	Method	Access	Description
`/resumes/me`	`GET`	Worker/ Admin	Fetch current worker's resume
`/resumes/me`	`PUT`	Worker/ Admin	Edit resume fields (headline, summary, experience)
`/resumes/generate-from-voice`	`POST`	Worker/ Admin	Generate an AI resume from the latest voice profile extraction
`/resumes/status`	`PATCH`	Worker/ Admin	Publish or archive resume
`/resumes/export-pdf`	`GET`	Worker/ Admin	Export resume as a downloadable PDF

11. Documents Module (`/documents`)

Endpoint	Method	Access	Description
`/documents/upload`	`POST`	Authenticated	Upload a document file (multipart `document` field, e.g., Aadhaar, PAN, certificates)
`/documents/mine`	`GET`	Authenticated	List current user's uploaded documents
`/documents/:id/verify`	`PATCH`	Admin	Verify or reject a document (admin only)

12. Saved Jobs Module (`/saved-jobs`)

All routes require authentication and `worker` or `admin` role.

Endpoint	Method	Access	Description
`/saved-jobs`	`POST`	Worker/Admin	Save/bookmark a job. Body: `{ jobId }`
`/saved-jobs/:jobId`	`DELETE`	Worker/Admin	Remove a saved job
`/saved-jobs/mine`	`GET`	Worker/Admin	List all saved jobs for the current worker

13. Shifts Module (`/shifts`)

All routes require authentication.

Endpoint	Method	Access	Description
`/shifts/schedule`	`POST`	Employer/ Admin	Schedule a shift for a hired worker. Body: `{ jobId, workerId, shiftDate, startTime, endTime }`
`/shifts/:shiftId/check-in`	`POST`	Worker	Record check-in for a scheduled shift (sets `checkInAt` timestamp)
`/shifts/:shiftId/check-out`	`POST`	Worker	Record check-out for a shift (sets `checkOutAt` timestamp)
`/shifts/missed`	`GET`	Admin	List missed shifts (scheduled but no check-in)

14. Payments Module (`/payments`)

Endpoint	Method	Access	Description
`/payments/razorpay/webhook`	`POST`	**Public**	Razorpay webhook callback (no auth header — verified by Razorpay signature)
`/payments/record-offline`	`POST`	Authenticated	Record a cash or UPI payment made outside the platform
`/payments/razorpay/order`	`POST`	Authenticated	Create a Razorpay order for online payment
`/payments/razorpay/verify`	`POST`	Authenticated	Verify Razorpay payment signature after client-side checkout
`/payments/history`	`GET`	Authenticated	Fetch payment history for the current user
`/payments/:paymentId/mark-paid`	`PATCH`	Employer/ Admin	Manually mark a payment as paid (offline/cash confirmation)
`/payments/:paymentId/refund`	`POST`	Admin	Process a refund for a payment

Note: The Razorpay webhook endpoint is registered **before** the `verifyJWT` middleware to allow Razorpay's servers to call it directly without a bearer token. Authentication is via Razorpay webhook signature verification instead.

15. Ratings Module (`/ratings`)

All routes require authentication.

Endpoint	Method	Access	Description
`/ratings`	`POST`	Authenticated	Submit a rating. Body: `{ jobId, toUserId, rating (1–5), review }`. Validated via `submitRatingValidator`. Unique index `{ jobId, fromUserId, toUserId }` prevents duplicate ratings.
`/ratings/user/:userId`	`GET`	Authenticated	Get all ratings for a specific user. Validated via `userRatingsParamValidator`.

Rating submission updates the target user's `ratingAvg` on their `WorkerProfile` or `EmployerProfile` using an aggregation recalculation.

16. Disputes Module (`/disputes`)

All routes require authentication.

Endpoint	Method	Access	Description
`/disputes`	`POST`	Authenticated	Raise a dispute. Body: `{ jobId, againstUserId, reason }`. Status starts as `open`.
`/disputes/mine`	`GET`	Authenticated	List disputes raised by or against the current user
`/disputes/:disputeId/evidence`	`POST`	Authenticated	Upload evidence for a dispute (multipart `evidence` image file)
`/disputes/:disputeId/resolve`	`PATCH`	Admin	Resolve or reject a dispute. Body: `{ status: "resolved"

17. Chat Module (`/chat`)

All routes require authentication. There is deliberately **no** `authorizeRoles` middleware — a dual-role account can be the worker in one conversation and the employer in another. Per-conversation participant membership is the sole authority.

Endpoint	Method	Access	Description
`/chat/conversations`	`POST`	Authenticated	Create a new conversation or return existing one. Body: `{ jobId, workerId }`. Validated via `createConversationValidator`.
`/chat/conversations`	`GET`	Authenticated	List all conversations for the current user
`/chat/conversations/:conversationId/messages`	`GET`	Authenticated	List messages in a conversation. Query: `?after=<ISO>` for polling. Validated via `listMessagesValidator`.
`/chat/conversations/:conversationId/messages`	`POST`	Authenticated	Send a message. Body: `{ text }`. Validated via `sendMessageValidator`.
`/chat/conversations/:conversationId/read`	`PATCH`	Authenticated	Mark all messages in a conversation as read. Validated via `conversationIdValidator`.

Authorization: Every read and write re-checks that the requesting user is a participant in the conversation and that a valid hiring relationship (`Application` status `accepted` or `completed`) exists. A conversation ID on its own grants no access.

Transport: Plain HTTP with polling (`after` query parameter). The previously existing `POST /chat/voice-message` endpoint was removed — it uploaded audio to Cloudinary without any relationship check.

18. Notifications Module (`/notifications`)

All routes require authentication.

Endpoint	Method	Access	Description
`/notifications`	`GET`	Authenticated	List notifications for the current user. Language resolved via `languageMiddleware` — trilingual title/body is returned in the user's preferred language.
`/notifications/unread-count`	`GET`	Authenticated	Get count of unread notifications
`/notifications/:id/read`	`PATCH`	Authenticated	Mark a single notification as read (sets `readAt`)
`/notifications/read-all`	`PATCH`	Authenticated	Mark all notifications as read

19. Admin Module (`/admin`)

All routes require authentication **and** `admin` role.

Endpoint	Method	Access	Description
`/admin/dashboard-stats`	`GET`	Admin	Platform-wide dashboard statistics (user counts, job counts, revenue)
`/admin/users/:userId/block`	`PATCH`	Admin	Block a user account (prevents login)
`/admin/users/:userId/unblock`	`PATCH`	Admin	Unblock a previously blocked user
`/admin/users/:userId/flag`	`PATCH`	Admin	Flag a user for review (sets `isFlagged: true`)
`/admin/documents/:docId/verify`	`PATCH`	Admin	Verify or reject an uploaded identity document
`/admin/skills/:skillId/verify`	`PATCH`	Admin	Verify a worker's skill claim (sets `verified: true` on `WorkerSkill`)
`/admin/jobs/:jobId/moderate`	`PATCH`	Admin	Moderate a job posting (approve, flag, or remove)
`/admin/audit-logs`	`GET`	Admin	View admin action audit trail (`AdminAction` collection)

Appendix A: Employer-Initiated Hiring (formerly `/offers`)

`/offers` no longer exists. An employer asking a specific worker to take a job is the SAME `Application` document a worker creates by applying, distinguished by `initiatedBy: "employer"` and `status: "invited"`:

Method	Path	Who
`POST`	`/applications/invite`	Employer who owns the job
`GET`	`/applications/invitations`	The invited worker
`POST`	`/applications/:applicationId/respond`	The invited worker (`accepted` / `declined`)

This keeps **one** authoritative hiring path: the unique `{jobId, workerId}` index prevents a duplicate request and prevents a request coexisting with an application for the same pair, and accepting an invitation goes through the same atomic capacity reservation (`reserveWorkerSlot`) as an employer accepting an application. The removed `/offers` path did neither — it called an uncapped increment, and three workers accepting requests for a one-worker job drove `filledWorkers` to 3.

Appendix B: Legacy Matching Engine

`services/matching.service.js` (`calculateMatchScore`) is no longer used by any production route. All match scoring is done by the Python AI service via `services/aiService.client.js`. The legacy service remains in the repository for reference and is still covered by its own unit test in `tests/api.test.js`, pending an explicit decision to remove it.

Appendix C: Error Response Format

All error responses follow this consistent structure:

```
{
  "statusCode": 400,
  "success": false,
  "message": "Human-readable error description",
  "errors": []
}
```

In `development` mode, a `stack` field is also included. In `production`, stack traces are suppressed. Malformed ObjectID parameters (`CastError`) return `400` with message `"Malformed ID format"` rather than `500`.